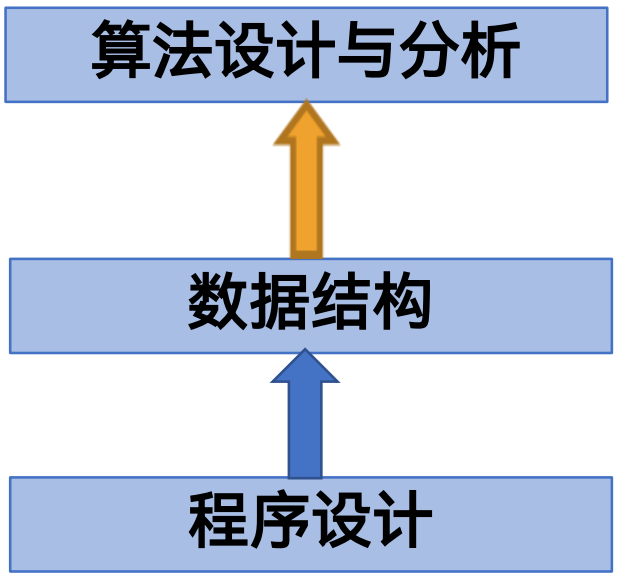


算法设计与分析

Algorithm Design and Analysis

计算机学院 潘永生
yspan@nwpu.edu.cn

《算法设计与分析》课程与《数据结构》和《程序设计》之间的关系。



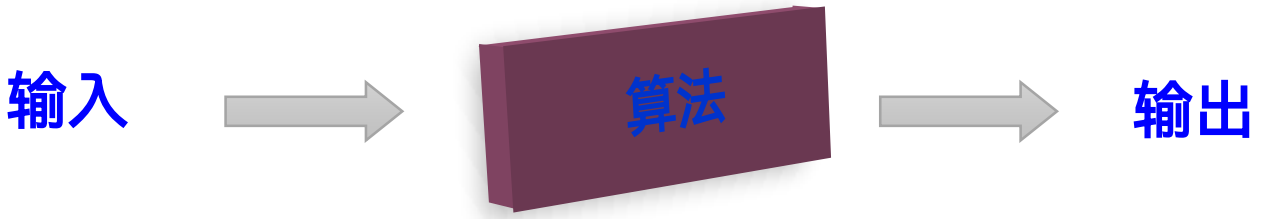


第一章 算法概论

Algorithm Introduction

什么是算法?

算法是求解问题的一系列计算步骤，用来将输入数据转换成输出结果：



如果一个算法对其每一个输入实例，都能输出正确的结果并停止，则称它是正确的。

算法设计应满足以下几条目标:

- 正确性
- 可使用性
- 可读性
- 健壮性
- 高效率与低存储量需求

1.1 算法的概念



特斯拉为何总是钟爱白色的卡车?



1.1 算法的概念

SEARCH

AMAZIN' BUT TRUE
A NEW YORK METS BASEBALL PODCAST

NEW YORK POST

Prince William asked to not walk beside Harry at Philip's funeral



Russia reportedly adds to military muscle in Black Sea region amid US...



'Consensual incest' should be decriminalized, advocates say



NEWS

Two killed in huge blaze after driverless Tesla crashes into tree

By Jorge Fitz-Gibbon

April 18, 2021 | 2:51pm | Updated



Elon Musk @elonmusk · 9小时

Your research as a private individual is better than professionals @WSJ!

Data logs recovered so far show Autopilot was not enabled & this car did not purchase FSD.

Moreover, standard Autopilot would require lane lines to turn on, which this street did not have.

1,059 3,451 1.3万

【例1.1】以下算法用于在带头结点的单链表*h*中查找第一个值为*x*的结点，找到后返回其逻辑序号（从1计起），否则返回0。分析该算法存在的问题。

```
int findx(LNode *h;int x)
{   LNode *p=h->next;
      int i=0;
      while (p->data!=x)
      {     i++;
            p=p->next;
      }
      return i;
}
```

解：问题：

- 当单链表中首结点值为x时，该算法返回0，此时应该返回逻辑序号1。
- 当单链表中不存在值为x的结点时，该算法执行出错，因为p为NULL时仍执行p=p->next。

所以该算法不满足正确性和健壮性。应改为：

```
int findx(LNode *h;int x)
{   LNode *p=h->next;
      int i=0;
      while (p->data!=x)
      {     i++;
            p=p->next;
      }
      return i;
}
```

```
int findx(LNode *h;int x)
{   LNode *p=h->next;
      int i=1;
      while (p!=NULL && p->data!=x)
      {     i++;
            p=p->next;
      }
      if (p==NULL)
            return 0;
      else
            return i;
}
```

算法具有以下5个重要特征：

- **有限性**
- **确定性**
- **可行性**
- **输入性**
- **输出性**

有限性
确定性
可行性
输入性
输出性

【例1.2】有下列两段描述：

描述1：

```
void exam1()
{  int n;
    n=2;
    while (n%2==0)
        n=n+2;
    printf("%d\n",n);
}
```

描述2：

```
void exam2()
{  int x,y;
    y=0;
    x=5/y;
    printf("%d,%d\n",x,y);
}
```

这两段描述均不能满足算法的特征，试问它们违反了算法的哪些特征？

解：（1）是一个死循环，违反了算法的有限性特征。（2）出现除零错误，违反了算法的可行性特征。

```
void exam1()
{  int n;
    n=2;
    while (n%2==0)
        n=n+2;
    printf("%d\n",n);
}
```

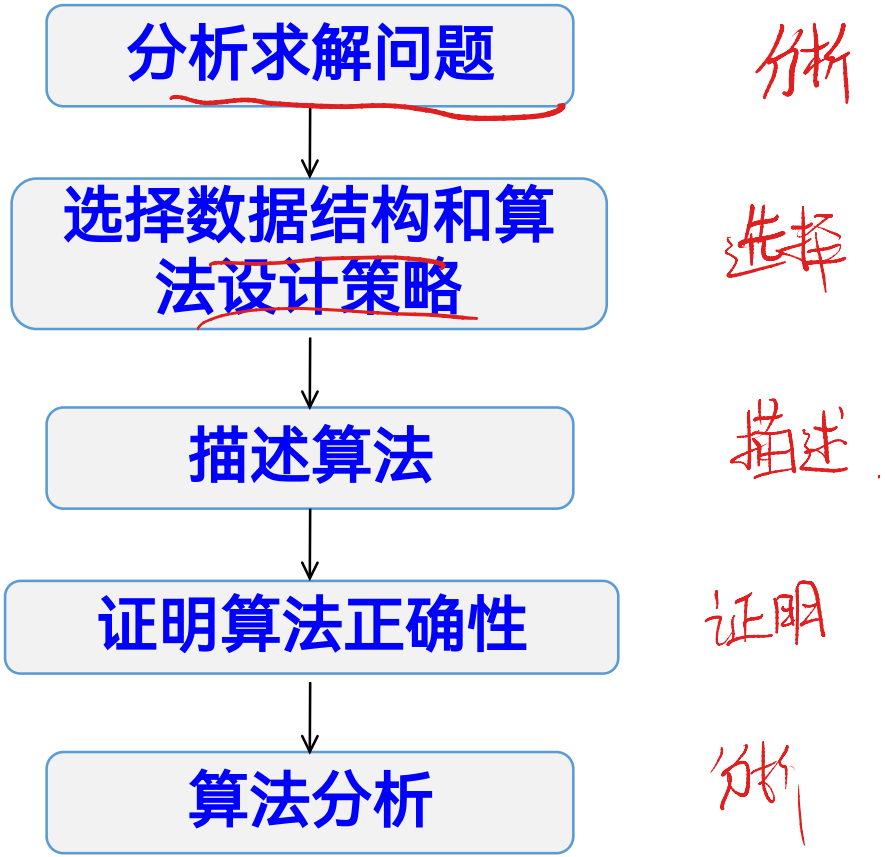
```
void exam2()
{  int x,y;
    y=0;
    x=5/y;
    printf("%d,%d\n",x,y);
}
```

算法与数据结构既有联系又有区别。

联系：数据结构是算法设计的基础。算法的操作对象是数据结构，在设计算法时，通常要构建适合这种算法的数据结构。数据结构设计主要是选择数据的存储方式，如确定求解问题中的数据采用数组存储还是采用链表存储等。算法设计就是在选定的存储结构上设计一个满足要求的好算法。

区别：数据结构关注的是数据的逻辑结构、存储结构以及基本操作，而算法更多的是关注如何在数据结构的基础上解决实际问题。算法是编程思想，数据结构则是这些思想的逻辑基础。

1.1 算法的概念



算法分析是分析算法占用计算机资源的情况。

所以算法分析的两个主要方面是分析算法的时间复杂度和空间复杂度。

1. 时间复杂度分析概述

一个算法是由控制结构（顺序、分支和循环3种）和原操作（指固有数据类型的操作）构成的，算法的运行时间取决于两者的综合效果。

```
bool Solve(double a[][MAX],int m,int n,double &s)
{
  int i; s=0;
  if (m!=n) return false;
  for (i=0;i<m;i++)
    s+=a[i][i];
  return true;
}
```

顺序结构

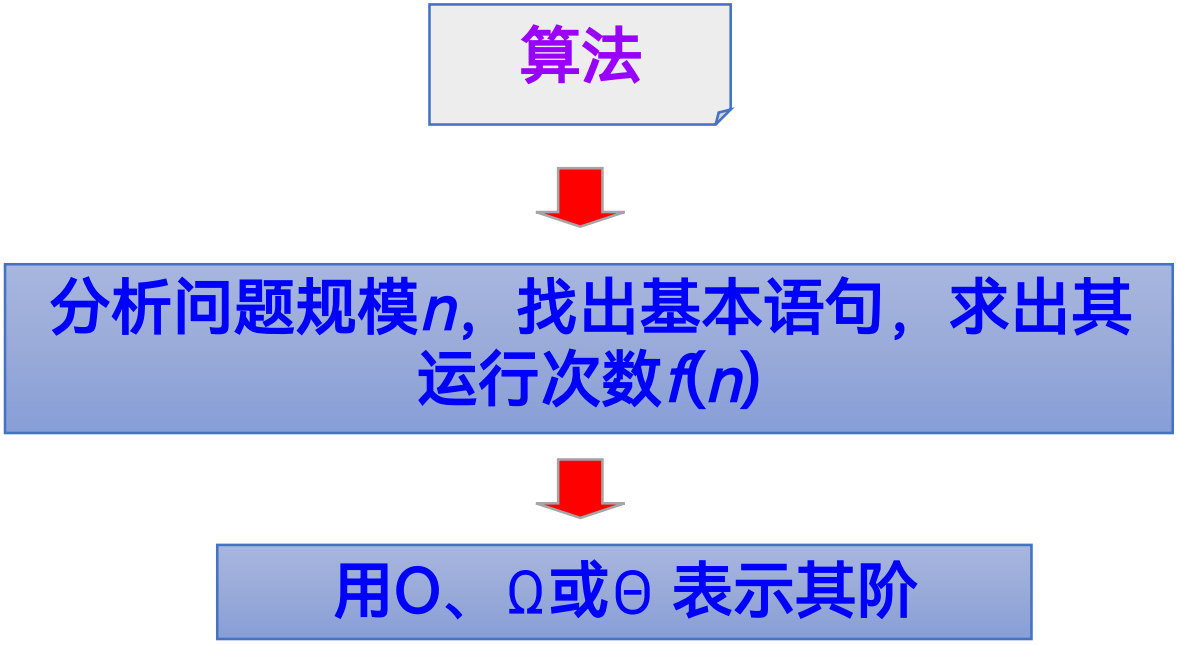
分支结构

循环结构

顺序结构

设n为算法中的问题规模，通常用大O、大Ω和Θ等三种渐进符号表示算法的执行时间与n之间的一种增长关系。

分析算法时间复杂度的一般步骤：



2. 渐进符号（O、Ω和Θ）

定义1（大O符号）， $f(n)=O(g(n))$（读作“$f(n)$是$g(n)$的大O”）当且仅当存在正常量c和n_0，使当$n \geq n_0$时，$f(n) \leq cg(n)$，即$g(n)$为$f(n)$的上界。

如$3n+2=O(n)$，因为当$n \geq 2$时，$3n+2 \leq 4n$。

$10n^2+4n+2=O(n^4)$，因为当$n \geq 2$时，$10n^2+4n+2 \leq 10n^4$。

大O符号用来描述增长率的上界，表示$f(n)$的增长最多像$g(n)$增长的那样快，也就是说，当输入规模为n时，算法消耗时间的最大值。这个上界的阶越低，结果就越有价值，所以，对于$10n^2+4n+2$，$O(n^2)$比$O(n^4)$有价值。

一个算法的时间用大O符号表示时，总是采用最有价值的$g(n)$表示，称之为“紧凑上界”或“紧确上界”。

一般地，如果$f(n)=a_m n^m+a_{m-1}n^{m-1}+\ldots+a_1 n+a_0$，有$f(n)=O(n^m)$。

Asymptotic Analysis: Big-oh

- Definition:

  For **T**(n) a non-negatively valued function, **T**(n) is in the set O($f(n)$) if there exist two positive constants c and n_0 such that **T**(n) <= $cf(n)$ for all $n > n_0$.

Usage: The algorithm is in O(n^2) in [best, average, worst] case.

Meaning: For all data sets big enough (i.e., $n>n_0$), the algorithm always executes in less than $cf(n)$ steps in [best, average, worst] case.

Big-oh notation indicates an upper bound.

Example: If $\mathbf{T}(n) = 3n^2$ then $\mathbf{T}(n)$ is in $O(n^2)$.

Wish tightest upper bound:
For example, while $\mathbf{T}(n) = 3n^2$ is in $O(n^3)$, we prefer $O(n^2)$.

Big-Oh Examples

Example 1: Finding value X in an array (average cost).

$\mathbf{T}(n) = c_s n/2$.
For all values of $n > 1$, $c_s n/2 <= c_s n$.
Therefore, by the definition, $\mathbf{T}(n)$ is in $O(n)$ for $n_0 = 1$ and $c = c_s$.

Big-Oh Examples

Example 2: $T(n) = c_1 n^2 + c_2 n$ in average case.

$c_1 n^2 + c_2 n <= c_1 n^2 + c_2 n^2 <= (c_1 + c_2) n^2$ for all $n > 1$.

$T(n) <= cn^2$ for $c = c_1 + c_2$ and $n_0 = 1$.

Therefore, $T(n)$ is in $O(n^2)$ by the definition.

Example 3: $T(n) = c$. We say this is in $O(1)$.

A Common Misunderstanding

"The best case for my algorithm is n=1 because that is the fastest."
WRONG!

Big-oh refers to a growth rate as n grows to ∞.

Best case is defined as which input of size n is cheapest among all inputs of size n.

定义2（大Ω符号）， $f(n)=\Omega(g(n))$（读作“$f(n)$是$g(n)$的大Ω”）当且仅当存在正常量c和n_0，使当$n\geq n_0$时，$f(n)\geq cg(n)$，即$g(n)$为$f(n)$的**下界**。

如$3n+2=\Omega(n)$，因为当$n\geq 1$时，$3n+2\geq 3n$。

$10n^2+4n+2=\Omega(n^2)$，因为当$n\geq 1$时，$10n^2+4n+2\geq n^2$。

大Ω符号用来描述增长率的下界，表示$f(n)$的增长最少像$g(n)$增长的那样快，也就是说，当输入规模为n时，算法消耗时间的最小值。

与大O符号对称，这个下界的阶越高，结果就越有价值，所以，对于$10n^2+4n+2$，$\Omega(n^2)$比$\Omega(n)$有价值。一个算法的时间用大Ω符号表示时，总是采用最有价值的$g(n)$表示，称之为“紧凑下界”或“紧确下界”。

一般地，如果$f(n)=a_m n^m+a_{m-1}n^{m-1}+\ldots+a_1 n+a_0$，有$f(n)=\Omega(n^m)$。

Big-Omega

Definition: For **T**(*n*) a non-negatively valued function, **T**(*n*) is in the set Ω(*g*(*n*)) if there exist two positive constants *c* and n_0 such that **T**(*n*) >= *cg*(*n*) for all $n > n_0$.

Meaning: For all data sets big enough (i.e.,$n > n_0$), the algorithm always executes in more than *cg*(*n*) steps.

Big-Omega notation indicates an lower bound.

Big-Omega Example

$\mathbf{T}(n) = c_1 n^2 + c_2 n.$

$c_1 n^2 + c_2 n >= c_1 n^2$ for all $n > 1$.
$\mathbf{T}(n) >= cn^2$ for $c = c_1$ and $n_0 = 1$.

Therefore, $\mathbf{T}(n)$ is in $\Omega(n^2)$ by the definition.

We want the greatest lower bound.

定义3（大Θ符号）， $f(n)=\Theta(g(n))$（读作“$f(n)$是$g(n)$的大Θ”）当且仅当存在正常量c_1、c_2和n_0，使当$n\geq n_0$时，有$c_1g(n)\leq f(n)\leq c_2g(n)$，即$g(n)$与$f(n)$的**同阶**。

如$3n+2=\Theta(n)$，$10n^2+4n+2=\Theta(n^2)$。

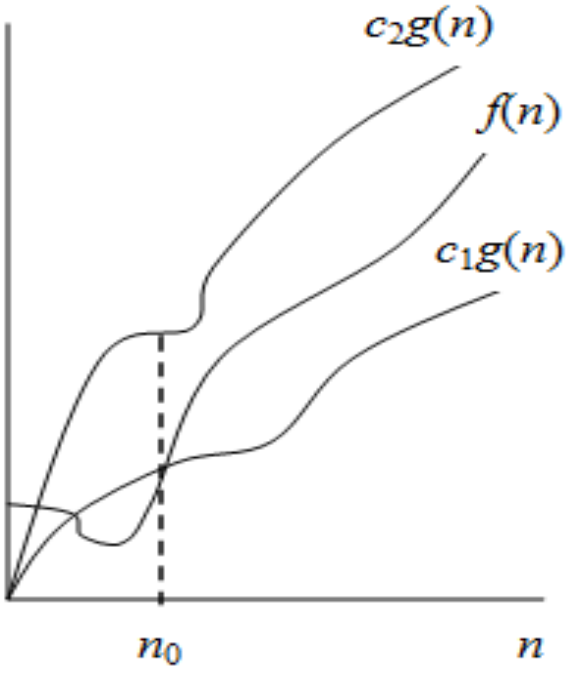
一般地，如果$f(n)=a_mn^m+a_{m-1}n^{m-1}+\ldots+a_1n+a_0$，有$f(n)=\Theta(n^m)$。

大Θ符号比大O符号和大Ω符号都精确，$f(n)=\Theta(g(n)$，当且仅当$g(n)$既是$f(n)$的上界又是$f(n)$的下界。

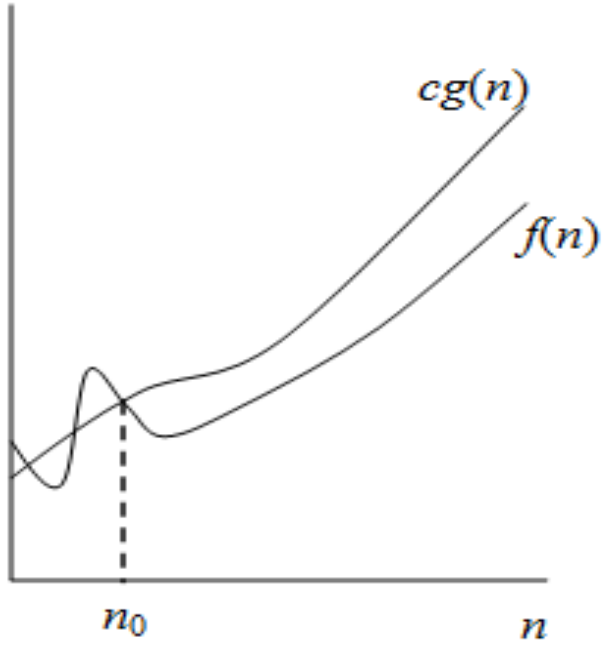
Theta Notation

When big-Oh and Ω meet, we indicate this by using Θ (big-Theta) notation.

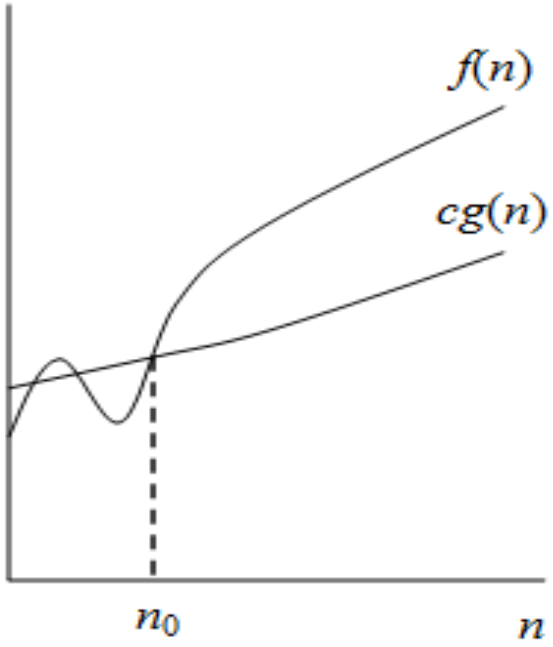
Definition: An algorithm is said to be Θ($h(n)$) if it is in O($h(n)$) and it is in Ω($h(n)$).



(a) $f(n)=\Theta(g(n))$



(b) $f(n)=O(g(n))$



(c) $f(n)=\Omega(g(n))$

A Common Misunderstanding

Confusing worst case with upper bound.

Upper bound refers to a growth rate.

Worst case refers to the worst input from among the choices for possible inputs of a given size.

Simplifying Rules

1. If $f(n)$ is in $O(g(n))$ and $g(n)$ is in $O(h(n))$, then $\boldsymbol{f(n)}$ is in $\boldsymbol{O(h(n))}$.

2. If $f(n)$ is in $O(kg(n))$ for any constant $k > 0$, then $\boldsymbol{f(n)}$ is in $\boldsymbol{O(g(n))}$.

3. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$, then $\boldsymbol{(f_1 + f_2)(n)}$ is in $\boldsymbol{O(\max(g_1(n), g_2(n)))}$.

4. If $f_1(n)$ is in $O(g_1(n))$ and $f_2(n)$ is in $O(g_2(n))$ then $\boldsymbol{f_1(n)f_2(n)}$ is in $\boldsymbol{O(g_1(n)g_2(n))}$.

Running Time Examples (1)

Example 1:

```
a = b;
```

This assignment takes constant time, so it is Θ(1).

Example 2:

```
sum = 0;
for (i=1; i<=n; i++)
  sum += n;
```

O(n).

Running Time Examples (2)

Example 3:

```
sum = 0;
for (j=1; j<=n; j++)
  for (i=1; i<=j; i++)
    sum++;

for (k=0; k<n; k++)
  A[k] = k;
```

$O(n^2)$.

Running Time Examples (3)

Example 4:

```
sum1 = 0;
for (i=1; i<=n; i++)
  for (j=1; j<=n; j++)
    sum1++;

sum2 = 0;
for (i=1; i<=n; i++)
  for (j=1; j<=i; j++)
    sum2++;
```

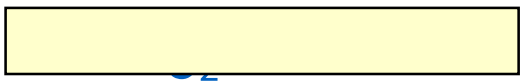
$O(n^2)$.

Running Time Examples (4)

Example 5:

```
sum1 = 0;
for (k=1; k<=n; k*=2)
  for (j=1; j<=n; j++)
    sum1++;

sum2 = 0;
for (k=1; k<=n; k*=2)
  for (j=1; j<=k; j++)
    sum2++;
```



Other Control Statements

while loop: Analyze like a **for** loop.

if statement: Take greater complexity of **then/else** clauses.

switch statement: Take complexity of most expensive case.

Subroutine call: Complexity of the subroutine.

【例】分析以下算法的时间复杂度：

```
void fun(int n)
{  int s=0,i,j,k;
   for (i=0;i<=n;i++)
          for (j=0;j<=i;j++)
              for (k=0;k<j;k++)
                    s++;
}
```

$f(n)=\sum_{i=0}^{n}\sum_{j=0}^{i}\sum_{0}^{j-1}1=\sum_{i=0}^{n}\sum_{j=0}^{i}(j)$

$(n+1)^3=n^3+3n^2+3n+1$
$(n+1)^3-n^3=3n^2+3n+1$
$n^3-(n-1)^3=3(n-1)^2+3(n-1)+1$
...
$3^3-2^3=3*2^2+3*2+1$
$2^3-1^3=3*1^2+3*1+1$
两边分别相加得
$(n+1)^3-1^3$
$=3*(1^2+2^2+...+n^2)+3(1+2+...+n)+1*n$
$(n^3+3n^2+3n)-3n(n+1)/2-n=3Sn$
$3Sn=n(2n^2+3n+1)/2=n(n+1)(2n+1)/2$
$Sn=n(n+1)(2n+1)/6$

解：该算法的基本语句是s++，所以有：

$$f(n)=\sum_{i=0}^{n}\sum_{j=0}^{i}\sum_{k=0}^{j-1}1=\sum_{i=0}^{n}\sum_{j=0}^{i}(j-1-0+1)=\sum_{i=0}^{n}\sum_{j=0}^{i}j$$

$s_n=\sum_{i=0}^{n}i^2+\sum_{i=0}^{n}i$

$$=\sum_{i=0}^{n}\frac{i(i+1)}{2}=\frac{1}{2}(\sum_{i=0}^{n}i^2+\sum_{i=0}^{n}i)=\frac{2n^3+6n^2+4n}{12}=O(n^3)$$

$\frac{n(n+1)(2n+1)}{6}$

则该算法的时间复杂度为$O(n^3)$。

3. 算法的最好、最坏和平均情况

定义4 设一个算法的输入规模为n，D_n是所有输入的集合，任一输入$I \in D_n$，$P(I)$是I出现的概率，有$\sum P(I) = 1$，$T(I)$是算法在输入I下所执行的基本语句次数，则该算法的平均执行时间为：$A(n)=\sum P(I) * T(I)$。

也就是说算法的平均情况是指用各种特定输入下的基本语句执行次数的带权平均值。

算法的最好情况为：$G(n) = \min_{I \in Dn} T(I)$，是指算法在所有输入$I$下所执行基本语句的最少次数。

算法的最坏情况为：$W(n) = \max_{I \in Dn} T(I)$，是指算法在所有输入$I$下所执行基本语句的最大次数。

【例】采用顺序查找方法，在长度为n的一维实型数组$a[0..n-1]$中查找值为x的元素。即从数组的第一个元素开始，逐个与被查值x进行比较。找到后返回1，否则返回0，对应的算法如下：

```
int Find(double a[],int n,double x)
{  int i=0;
   while (i<n)
   {  if (a[i]==x) break;
      i++;
   }
   if (i<n) return 1;
   else return 0;
}
```

回答以下问题：

（1）分析该算法在等概率情况下成功查找到值为x的元素的最好、最坏和平均时间复杂度。

（2）假设被查值x在数组a中的概率是q，求算法的时间复杂度。

```
int Find(double a[],int n,double x)
{  int i=0;
   while (i<n)
   {  if (a[i]==x) break;
      i++;
   }
   if (i<n) return 1;
   else return 0;
}
```

解：（1）算法的while循环中的if语句是基本语句。*a*数组中有*n*个元素，当第一个元素*a*[0]等于*x*，此时基本语句仅执行一次，此时呈现最好的情况，即$G(n)=O(1)$。

当*a*中最后一个元素*a*[*n*-1]等于*x*，此时基本语句执行*n*次，此时呈现最坏的情况，即$W(n)=O(n)$。

```
int Find(double a[],int n,double x)
{  int i=0;
   while (i<n)
   {  if (a[i]==x) break;
      i++;
   }
   if (i<n) return 1;
   else return 0;
}
```

对于其他情况，假设查找每个元素的概率相同，则$P(a[i])=1/n$（$0\le i\le n-1$），而成功找到$a[i]$元素时基本语句正好执行$i+1$次，所以：

$$A(n)=\sum_{i=0}^{n-1}\frac{1}{n}(i+1)=\frac{1}{n}\sum_{i=0}^{n-1}(i+1)=\frac{n+1}{2}$$

$=O(n)$。

（2）当被查值x在数组a中的概率为q时，算法执行有$n+1$种情况，即n种成功查找和一种不成功查找。

对于成功查找，假设是等概率情况，则元素$a[i]$被查找到的概率$P(a[i])=q/n$，成功找到$a[i]$元素时基本语句正好执行$i+1$次。

对于不成功查找，其概率为$1-q$，不成功查找时基本语句正好执行n次。

所以：

$$A(n)=\sum_{I\in D_n} P(I)*T(I)$$

$$=\sum_{i=0}^{n} P(I)*T(I)=\sum_{i=0}^{n-1}\frac{q}{n}(i+1)+(1-q)n=\frac{(n+1)q}{2}+(1-q)n$$

如果已知需要查找的x有一半的机会在数组中，此时$q=1/2$，则

$A(n)=[(n+1)/4]+n/2\approx 3n/4$。

4. 非递归算法的时间复杂度分析

对于非递归算法，分析其时间复杂度相对比较简单，关键是求出代表算法执行时间的表达式。

通常是算法中基本语句的执行次数，是一个关于问题规模n的表达式，然后用渐进符号来表示这个表达式即得到算法的时间复杂度。

【例1】给出以下算法的时间复杂度。

```
void func(int n)
{   int i=1,k=100;
    while (i<=n)
    {  k++;
       i+=2;
    }
}
```

解：算法中基本语句是while循环内的语句。设while循环语句执行的次数为m，i从1开始递增，最后取值为$1+2m$，有：

$i=1+2m\leq n$

$f(n)=m\leq(n-1)/2=O(n)$。

该算法的时间复杂度为$O(n)$。

5. 递归算法的时间复杂度分析

递归算法是采用一种分而治之的方法，把一个“大问题”分解为若干个相似的“小问题”来求解。

对递归算法时间复杂度的分析，关键是根据递归过程建立递推关系式，然后求解这个递推关系式，得到一个表示算法执行时间的表达式，最后用渐进符号来表示这个表达式即得到算法的时间复杂度。

【例】有以下递归算法：

```
void mergesort(int a[],int i,int j)
{   int m;
    if (i!=j)
    {         m=(i+j)/2;
        mergesort(a,i,m);
        mergesort(a,m+1,j);
        merge(a,i,j,m);
    }
}
```

其中，mergesort()用于数组$a[0..n-1]$（设$n=2^k$，这里的k为正整数）的归并排序，调用该算法的方式为：

mergesort(a，0，$n-1$);

另外merge(a，i，j，m)用于两个有序子序列$a[i..j]$和$a[j+1..m]$的有序合并，是非递归函数，它的时间复杂度为O(n)（这里$n=j-i+1$）。分析上述调用的时间复杂度。

解：设调用mergesort(*a*，0，*n*-1)的执行时间为*T*(*n*)，由其执行过程得到以下求执行时间的递归关系（递推关系式）：

$$T(n)=O(1) \quad \text{当} n=1$$
$$T(n)=2T(n/2)+O(n) \quad \text{当} n>1$$

```
void mergesort(int a[],int i,int j)
{   int m;
    if (i!=j)
    {   m=(i+j)/2;
        mergesort(a,i,m);
        mergesort(a,m+1,j);
        merge(a,i,j,m);
    }
}
```

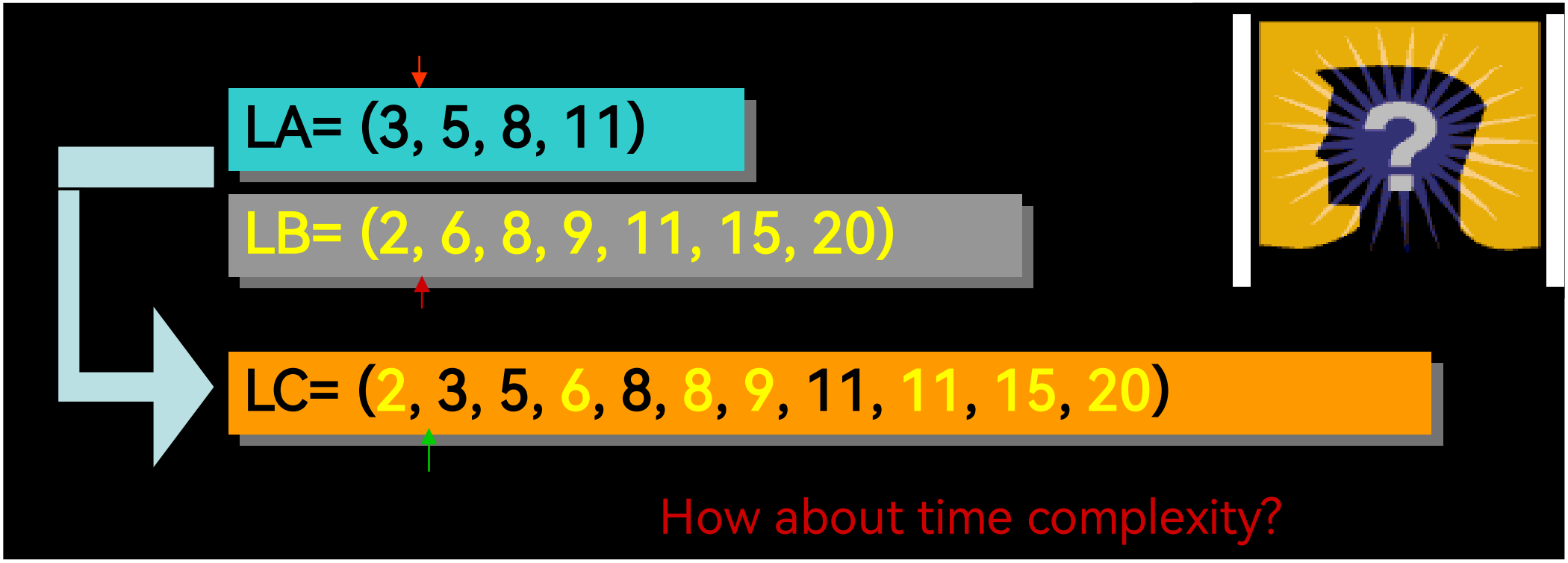
其中，O(*n*)为merge()所需的时间，设为*cn*（*c*为正常量）。因此：

$$\begin{aligned}T(n) &= 2T(n/2)+cn=2[2T(n/2^2)+cn/2]+cn=2^2T(n/2^2)+2cn\\ &= 2^3T(n/2^3)+3cn\\ &= \cdots\\ &= 2^kT(n/2^k)+kcn\\ &= nO(1)+cn\log_2 n=n+cn\log_2 n \text{ //这里假设} n=2^k\text{，则} k=\log_2 n\\ &= O(n\log_2 n)\end{aligned}$$

Sorted lists merging

Demo

- Merge two sorted lists into a new list and the new one is also sorted as before.



Implementation: Sorted lists merging

```
void MergeSqList ( SeqList *la, SeqList *lb, SeqList *lc)
{
    int i=0, j=0, k=0;
    m = la->length; n = lb->length; lc->length=0;
    while ( i < m && j<n) {
        x = retrieve_seq (la, i);        // Get an item x from la
        y = retrieve_seq (lb, j);        // Get an item y from lb
        if (x<=y) {
            Insert_Seq(lc, x, k); i++; k++;  // Insert x into lc
        }
        else {
            Insert_Seq(lc, y, k); j++; k++;  // Insert y into lc
        }
    }
    while (i<m) {x = retrieve_seq (la, i); Insert_Seq(lc, x, k); i++; k++;}
    while (j<n) {y = retrieve_seq (lb, j); Insert_Seq(lc, y, k); j++; k++;}
}
```

【例】求解梵塔问题的递归算法如下，分析其时间复杂度。

```
void Hanoi(int n,char x,char y,char z)
{  if (n==1)
      printf("将盘片%d从%c搬到%c\n",n,x,z);
   else
   {   Hanoi(n-1,x,z,y);
       printf("将盘片%d从%c搬到%c\n",n,x,z);
             Hanoi(n-1,y,x,z);
   }
}
```

解：设调用Hanoi(*n*，*x*，*y*，*z*)的执行时间为*T*(*n*)，由其执行过程得到以下求执行时间的递归关系（递推关系式）：

$T(n)=O(1)$ 当$n=1$

$T(n)=2T(n-1)+1$ 当$n>1$

```
void Hanoi(int n,char x,char y,char z)
{  if (n==1)
      printf("将盘片%d从%c搬到%c\n",n,x,z);
   else
   {   Hanoi(n-1,x,z,y);
       printf("将盘片%d从%c搬到%c\n",n,x,z);
             Hanoi(n-1,y,x,z);
   }
}
```

$$T(n) = 2[2T(n-2)+1]+1=2^2T(n-2)+1+2^1$$
$$= 2^3T(n-3)+1+2^1+2^2$$
$$= \dots$$
$$= 2^{n-1}T(1)+1+2^1+2^2+\dots+2^{n-2}$$
$$= 2^n-1 = O(2^n)$$

$T(n)=O(1)$

$T(n)=2T(n-1)+1$

$\frac{1\times(1-2^{n-1})}{1-2}$

$2^{n-1}-1+2^{n-1}$

主方法

主方法

主方法（master method）提供了解如下形式递归方程的一般方法：

$$T(n) = aT(n/b) + f(n)$$

$T(n) = aT(\frac{n}{b}) + f(n)$

$T(n) = aT(n/b) + f(n)$

其中$a \geq 1$，$b > 1$为常数，该方程描述了算法的执行时间，算法将规模为n的问题分解成a个子问题，每个子问题的大小为n/b。

例如，对于递归方程 $T(n) = 3T(n/4) + n^2$，有：$a=3$，$b=4$，$f(n)=n^2$。

$T(n) = aT(\frac{n}{b}) + f(n)$

base $= \log_b^a$

主定理：设$a \geq 1$，$b > 1$为常数，$f(n)$为一个函数，$T(n)$由（2.11）的递归方程定义，其中n为非负整数，则$T(n)$计算如下：

(1) 若对某些常数$\varepsilon > 0$，有$f(n) = O(n^{\log_b a - \varepsilon})$，那么$T(n) = O(n^{\log_b a})$。

(2) 若$f(n) = O(n^{\log_b a})$，那么$T(n) = O(n^{\log_b a} \log n)$。

(3) 若对某些常数$\varepsilon > 0$，有$f(n) = O(n^{\log_b a + \varepsilon})$，并且对常数$c<1$与所有足够大的$n$，有$af(n/b) \leq cf(n)$，那么$T(n) = O(f(n))$。

$T(n) = aT(\frac{n}{b}) + f(n)$

应用该定理的过程是，首先把函数$f(n)$与函数进行比较，递归方程的解由这两个函数中较大的一个决定：

情况（1），函数$n^{\log_b a}$比函数$f(n)$更大，则$T(n)=O(n^{\log_b a})$。

情况（2），函数$n^{\log_b a}$和函数$f(n)$一样大，则$T(n)=O(n^{\log_b a}\log n)$。

情况（3），函数$n^{\log_b a}$比函数$f(n)$小，则$T(n)=O(f(n))$。

分析以下递归方程的时间复杂度：

$T(n)=1$ 当 $n=1$

$T(n)=4T(n/2)+n$ 当 $n>1$

$T(n)=4T(\frac{n}{2})+n$

解：这里$a=4$，$b=2$，$f(n)=n$。

$b=2$
$a=4$

$f(n)=$

因此，$n^{\log_2 4}=n^2$，比$f(n)$大，满足情况（1），

$n^{\log_b a}$ n^2 $\geq f(n)$

所以$T(n)=O(n^{\log_b a})=O(n^2)$。

$T(n)=1$ 当 $n=1$

$T(n)=2T(n/2)+n^2$ 当 $n>1$

解：这里$a=2$，$b=2$，$f(n)=n^2$。因此，$n^{\log_2 2}=n$，比$f(n)$小，满足情况(3)，所以$T(n)=O(f(n))=O(n^2)$，与采用递归树的结果相同。

一个算法的存储量包括形参所占空间和临时变量所占空间。在对算法进行存储空间分析时，只考察临时变量所占空间。

例如，有以下算法，其中临时空间为变量*i*、maxi占用的空间。所以，空间复杂度是对一个算法在运行过程中临时占用的存储空间大小的量度，一般也作为问题规模n的函数，以数量级形式给出，记作：

$S(n)=O(g(n))$、$\Omega(g(n))$或$\Theta(g(n))$

其中渐进符号的含义与时间复杂度中的含义相同。

```
int max(int a[], int n)
{  int i, maxi=0;
   for (i=1;i<n;i++)
          if (a[i]>a[maxi])
              maxi=i;
   return a[maxi];
}
```

函数体内分配的变量空间为临时空间，不计形参占用的空间，这里的仅计*i*、maxi变量的空间，其空间复杂度为O(1)。

算法空间复杂度的分析方法与前面介绍的时间复杂度分析方法相似。

【例】分析算法的空间复杂度。

```
void func(int n)
{   int i=1,k=100;
    while (i<=n)
    {          k++;
               i+=2;
    }
}
```

解：该算法是一个非递归算法，其中只临时分配了*i*、*k*两个变量的空间，它与问题规模*n*无关，所以其空间复杂度均为O(1)，即该算法为原时工作算法。

【例】有如下递归算法，分析调用maxelem(*a*，0，*n*-1)的空间复杂度。

```
int maxelem(int a[],int i,int j)
{   int mid=(i+j)/2,max1,max2;
    if (i<j)
    {       max1=maxelem(a,i,mid);
            max2=maxelem(a,mid+1,j);
            return (max1>max2)?max1:max2;
    }
    else return a[i];
}
```

解： 执行该递归算法需要多次调用自身，每次调用只临时分配3个整型变量的空间（O(1)）。

设调用maxelem(*a*，0，*n*-1)的空间为*S*(*n*)，有：

```
int maxelem(int a[],int i,int j)
{  int mid=(i+j)/2,max1,max2;
   if (i<j)
   {        max1=maxelem(a,i,mid);
            max2=maxelem(a,mid+1,j);
            return (max1>max2)?max1:max2;
   }
   else return a[i];
}
```

$S(n)=O(1)$ 当$n=1$

$S(n)=2S(n/2)+O(1)$ 当$n>1$

则：$S(n)=2S(n/2)+1=2[2S(n/2^2)+1]+1=2^2S(n/2^2)+1+2^1$

$=2^3S(n/2^3)+1+2^1+2^2$

$=\ldots$

$=2^kS(n/2^k)+1+2^1+2^2+\ldots+2^{k-1}$（设$n=2^k$，即$k=\log_2 n$）

$=2^k+2^k-1=2n-1=O(n)$